Yesterday 8:49 PM

DP_Csestudy.pdf
PDF

https://github.com/Oracledelphio/Design-Pattern-Casestudy.git
I am doing a Design Pattern Case Study for my 23CSE455 course.

I have uploaded:
1. The official Design Pattern Case Study Instructions PDF.
2. The complete source code of my assigned application as a ZIP file.

First, do NOT identify design patterns yet.

Your task is only to understand and document the application.

Please carefully inspect the complete source code and give me:

1. Application name and purpose
2. Programming language(s) and framework(s)
3. Complete repository/project structure
4. Important source-code directories
5. Important classes, interfaces, modules, controllers, models, services, utilities, etc.
6. Main execution flow
7. Important dependencies
8. Database/storage components if any
9. Existing tests and how they are executed
10. Configuration/environment requirements
11. Any important architectural components
12. Files/modules that will probably be relevant for design-pattern analysis

IMPORTANT:
- Do not identify or claim any design pattern yet.
- Do not invent classes, relationships, or functionality.
- Base everything strictly on the uploaded source code.

- If something cannot be determined from the source, explicitly say "Not confirmed from source".
- Give exact file paths wherever possible.
- Do not modify the source code.

At the end, provide a concise "Source Inspection Map" showing which files/modules should be examined closely in Part 1.

Worked for 53s

Design Pattern Case Study — Part 1: Application Source Inspection

I'll keep this stage strictly focused on **understanding and documenting the existing application**. I will not identify design patterns, recommend refactoring, or claim relationships that the source does not establish.

Source verification and scope

I was able to inspect an existing source snapshot stored in your previous `A1_submission_ready.zip` artifact. It contains the original Rails application under `A1/data/code/original/`, including its controllers, models, views, database files, configuration, and tests.

There is one important limitation: the ZIP you said you uploaded is not attached to this conversation, and the GitHub URL you supplied returned a 404 when I checked it. Therefore, I cannot confirm that the archived snapshot corresponds to the exact current state of that GitHub repository or establish its original commit.

The analysis below is grounded in the accessible source snapshot, not assumptions about the inaccessible repository. No source code has been modified.

1. Application name and purpose

Rails Room Booking

Application identity taken from the source README and Rails configuration

The application is a web-based hotel room and vacation-rental booking system. It supports property listings, reservations, user accounts, configurable VAT, online payments, and booking-related webhook integration.

Primary source files

- `README.md`
- `config/application.rb`
- `config/routes.rb`
- `app/models/room.rb`
- `app/models/booking.rb`

The README describes the following features:

- User registration, login, and password recovery.
- Creating, editing, and managing accommodation listings.
- Booking accommodations for date ranges.
- Managing account settings, including VAT.
- Online payment integration through Stripe via Payify.
- Webhook integration through Integromat.
- English and French interface translations.

The README also describes synchronization with Airbnb as an upcoming feature. Its implementation is **not confirmed from source**.

2. Programming languages and frameworks

Technology	Evidence and purpose
Ruby	Main application language; `.ruby-version` specifies Ruby 2.7.5.
Ruby on Rails	Web application framework; `Gemfile` specifies `rails ~> 7.0.4`, while `Gemfile.lock` resolves Rails 7.0.6.
ERB	HTML templates in `app/views/`.
JavaScript	Client-side functionality under `app/javascript/`.
SCSS	Stylesheets under `app/assets/stylesheets/`.
SQL / PostgreSQL	Database configured through Active Record and `pg`.
RSpec	Test framework, configured in `spec/`.
Docker	Container configuration in `Dockerfile` and `docker-compose.yml`.

The application uses the conventional Rails directory organization, with application code, database migrations, configuration, public assets, and tests in separate directories.

3. Repository and project structure

The following tree summarizes the inspected application. It includes the top-level files and the complete major application-module organization, rather than expanding every static image and generated asset.

Plain text

```
original/
├── .env.dist
├── .gitattributes
├── .gitignore
├── .ruby-version
├── .rspec
├── .rubocop.yml
├── Dockerfile
├── docker-compose.yml
├── Gemfile
├── Gemfile.lock
├── README.md
├── Rakefile
├── config.ru
├── bin/
│   ├── bundle
│   ├── dev
│   ├── importmap
│   ├── rails
│   ├── rake
│   └── setup
```

The tree above is a functional overview. Some directories contain additional nested files, including locale translation files, image assets, environment-specific configuration, migrations, and view partials.

4. Important source-code directories

Directory	Responsibility established by its contents
`app/controllers/`	Receives and processes web requests.
`app/controllers/concerns/`	Contains shared controller functionality.
`app/models/`	Contains the main application data models and their business rules.
`app/models/concerns/`	Contains shared model functionality related to ownership, pricing, and settings.

`app/policies/`	Contains authorization checks.
`app/views/`	Contains ERB templates rendered by the application.
`app/javascript/`	Contains JavaScript entry points, Stimulus controllers, and translation utilities.
`app/assets/`	Contains stylesheets, images, and asset-pipeline files.
`config/`	Defines routes, database access, application settings, initializers, and environment configuration.
`db/migrate/`	Contains database schema migrations.
`db/`	Contains the current schema and seed instructions/data.
`spec/`	Contains RSpec tests, factories, and test configuration.
`bin/`	Contains executable project commands.
`public/`	Contains static public assets and error pages.
`lib/`	Contains asset and task directories; substantial custom business logic here is not confirmed.
`vendor/`	Contains placeholder directories for vendored assets.

5. Important classes, modules, controllers, and models

All paths in this section are relative to `A1/data/code/original/` in the archived source snapshot.

5.1 Controllers

`app/controllers/application_controller.rb`

Base controller. Sets the layout, includes authorization and shared controller concerns, and runs a `before_action` for the current scope.

`app/controllers/home_controller.rb`

Defines `index`. The root route's `language_redirection` action is supplied by an included concern rather than defined in this controller.

`app/controllers/rooms_controller.rb`

Configures room operations, filters listings for the current user when the `my` scope is requested, permits room form fields, and prepares a booking form for a room's show action.

`app/controllers/bookings_controller.rb`

Handles booking operations, cancellation, user-scoped booking queries, and payment-intent initialization when showing a booking.

`app/controllers/settings_controller.rb`

Handles user-specific settings, permits the VAT field, and redirects successful operations to the settings page.

`app/controllers/payify/payments_controller.rb`

Includes the Payify payment-controller concern and associates the payment completion action with its booking model.

5.2 Shared controller modules

File	Observed responsibility
`app/controllers/concerns/crud_concern.rb`	Defines shared index, show, new, create, edit, update, and destroy actions; handles authorization, form parameters, pagination, record lookup, and redirects.
`app/controllers/concerns/auth_concern.rb`	Defines `set_current_scope`, which assigns the current user's ID to `CurrentScope.user_id` when a user is signed in.
`app/controllers/concerns/locale_concern.rb`	Determines the locale, sets the language cookie, switches locale for a request, and prepares language-switching paths.
`app/controllers/concerns/page_concern.rb`	Sets page title, description, URL, and language variables.

`CrudConcern` is especially important to understand because several controllers include it. The actual class selected by its abstract methods differs between controllers.

5.3 Models and shared model modules

File	Responsibilities visible in source

`app/models/application_record.rb`	Base Active Record class for application models.
`app/models/user.rb`	User authentication configuration, associations with rooms and bookings, administrator status, and access to the current user through `CurrentScope`.
`app/models/room.rb`	Room attributes, user and booking associations, image attachments, room type, validations, sorting, enabled amenities, and booked-date retrieval.
`app/models/booking.rb`	Booking associations, date validations, availability checks, price/payment-related methods, cancellation, and booking webhook notification.
`app/models/setting.rb`	Associates settings with a user.
`app/models/concerns/has_owner_concern.rb`	Assigns an owner before validation when no `user_id` is present and a current user is available.
`app/models/concerns/has_price_concern.rb`	Calculates booking prices before saving when the start or end date changes.
`app/models/concerns/has_settings_concern.rb`	Defines a settings association and creates settings after a user is created.

The booking model includes the Payify payment concern as well as the two application-defined model concerns for ownership and pricing.

5.4 Authorization policies

- `app/policies/application_policy.rb` defines the base authorization methods and a nested `Scope` class.
- `app/policies/room_policy.rb` adds an ownership check for updating a room.
- `app/policies/booking_policy.rb` adds an ownership check for updating and cancelling a booking.
- `app/policies/setting_policy.rb` adds an ownership check for updating settings.

The controllers invoke authorization through Pundit. The exact runtime authorization outcome depends on the record and current user.

5.5 Other modules

- `app/helpers/application_helper.rb` — application view helpers.
- `app/jobs/application_job.rb` — base job class.
- `app/mailers/application_mailer.rb` — base mailer class.

- `app/channels/application_cable/connection.rb` and `channel.rb` — Action Cable connection and channel base classes.
- `app/javascript/controllers/datepicker_controller.js` — date-picker-related client functionality.
- `app/javascript/controllers/image_uploader_controller.js` — image-upload-related client functionality.
- `app/javascript/controllers/modal_controller.js` — modal-related client functionality.

The presence of these base classes and modules does not by itself confirm that every associated feature is actively used in the main application flow.

6. Main execution flow

The following is the request flow supported by the routes, controllers, and models. It is a functional flow diagram, not a design-pattern diagram.

Detailed execution sequence

1. **Routing:** `config/routes.rb` sends `/` to `home#language_redirection`. Locale-scoped routes provide access to rooms, bookings, settings, and Devise user routes. Payify's engine is mounted separately at `/payify`.

2. **Controller setup:** `ApplicationController` configures the base layout and includes authorization, page, locale, and current-scope functionality.
3. **Resource handling:** The appropriate controller invokes shared CRUD actions or implements its own additional behavior.
4. **Data access:** Controllers retrieve or construct Active Record models such as `Room`, `Booking`, and `Setting`.
5. **Validation:** Model validations are applied before valid records are saved. Booking validation checks the date order and overlapping reservations for a room.
6. **Persistence and callbacks:** Booking model callbacks initiate payment-record creation and booking notification behavior.
7. **Rendering or redirection:** Controllers render the corresponding views or redirect after successful operations.

The exact full lifecycle of a payment, including all callbacks inside the third-party Payify gem, is **not confirmed from application source alone**.

7. Important dependencies

The versions below are taken from `Gemfile` and `Gemfile.lock`, rather than inferred from current online documentation.

Dependency	Locked version or declaration	Observed purpose
Rails	7.0.6 locked; `~> 7.0.4` in Gemfile	Web application framework.
Ruby	2.7.5 in `.ruby-version`	Application runtime requirement.
PostgreSQL adapter (`pg`)	1.5.3	Database connectivity.
Puma	5.6.6	Web server.
Devise	4.9.2	User authentication.
Pundit	2.3.0	Authorization.
Kaminari	1.2.2	Pagination.
Payify	0.1.18	Payment integration.
Integromat	0.1.0	Webhook integration.
Bootstrap	5.1.3	UI styling framework.
RSpec Rails	6.0.3	Rails testing support.

Factory Bot Rails	Declared in Gemfile	Test data factories.
Faker	Declared in Gemfile	Test-data generation support.
Capybara	Declared in Gemfile	Browser/system testing support.
Selenium WebDriver	Declared in Gemfile	Browser automation support.
Image Processing	`~> 1.2`	Image-processing support.
Importmap Rails	`~> 1.1`	JavaScript import-map integration.
Turbo Rails	Declared in Gemfile	Turbo integration.
Stimulus Rails	Declared in Gemfile	Stimulus integration.
Sass / CSS tooling	Sass-related Rails dependencies	Stylesheet processing.
Money	`~> 6.16`	Monetary-value functionality.
Dotenv Rails	`~> 2.8.1`	Environment-variable loading support.

Other declared dependencies include `jsbundling-rails`, `cssbundling-rails`, `sprockets-rails`, `jbuilder`, `slug`, `i18n-js`, `listen`, `rails-i18n`, and `webdrivers`.

The exact installed versions of dependencies without a version shown in the table can be checked in `Gemfile.lock`. Their mere presence in the dependency file does not establish that every dependency is used by a specific execution path.

8. Database and storage

Database configuration

`config/database.yml` specifies PostgreSQL for development, testing, and production. It uses environment variables for connection details.

The inspected schema in `db/schema.rb` contains these application and supporting tables:

Table	Purpose inferred from schema columns
`users`	User accounts, encrypted passwords, and administrator flag.
`rooms`	Accommodation information, pricing, amenities, location, and owner ID.
`bookings`	Booking dates, guest count, room/user IDs, price totals, and cancellation information.

`settings`	User-specific settings, including VAT.
`payify_payments`	Payment status, payment method, transaction information, and payment-related fields.
`active_storage_attachments`	Links stored files to application records.
`active_storage_blobs`	Active Storage file metadata.
`active_storage_variant_records`	Active Storage image-variant records.

The schema is defined as `ActiveRecord::Schema[7.0]` with version `2023_06_18_133901`.

Relationships visible in the models

- A `User` has many `Room` records.
- A `User` has many `Booking` records.
- A `Room` belongs to a user and has many bookings.
- A `Booking` belongs to a user and a room.
- A `Setting` belongs to a user.
- A `Room` has many attached images.
- The user model includes settings-related functionality.

The schema's explicit foreign-key declarations are for Active Storage tables. Other relationships are expressed through the Active Record model associations and indexed ID columns; database-enforced foreign keys for those associations are not confirmed by the inspected schema.

Other storage

`config/storage.yml` defines local disk storage and test disk storage. Cloud storage configurations are present as commented examples. Whether any cloud storage service is configured in a deployed environment is **not confirmed from source**.

9. Existing tests and execution

The application contains two model-specification files:

File	What it tests
`spec/models/room_spec.rb`	Room creation and the expected room type.

`spec/models/booking_spec.rb`	Booking price calculations, required dates, invalid date ranges, and overlapping-booking validation.

Supporting files include:

- `spec/factories.rb`
- `spec/factories/rooms.rb`
- `spec/factories/bookings.rb`
- `spec/factories/users.rb`
- `spec/rails_helper.rb`
- `spec/spec_helper.rb`
- `spec/support/chrome.rb`
- `spec/support/factory_bot.rb`

Test commands

Run from the application root, `A1/data/code/original/` :

```bash
bundle install
bundle exec rspec
```

To run individual specifications:

```bash
bundle exec rspec spec/models/room_spec.rb
bundle exec rspec spec/models/booking_spec.rb
```

Test status: I have inspected the test files but have not executed the test suite in this inspection. The archived package's test-status note also says the original application tests were not run during its packaging audit. No passing test count or successful runtime result is claimed.

There are no controller or system specification files in the inspected `spec/` tree. Whether additional tests exist outside this snapshot is not confirmed.

10. Configuration and environment requirements

Runtime and database

- Ruby 2.7.5 is specified by `.ruby-version`.
- Rails is specified in `Gemfile`.
- PostgreSQL is required by `config/database.yml`.
- Bundler is used to install and resolve dependencies.
- The application can be started using `bin/rails` or the documented Rails commands.
- Docker configuration is provided through `Dockerfile` and `docker-compose.yml`.

Environment variables

Variable	Use visible in source
`DATABASE_USERNAME`	PostgreSQL username.
`DATABASE_PASSWORD`	PostgreSQL password.
`DATABASE_HOST`	PostgreSQL hostname.
`DATABASE_PORT`	PostgreSQL port; defaults to 5432 in `database.yml`.
`RAILS_MAX_THREADS`	Database connection-pool size; defaults to 5.
`STRIPE_API_KEY`	Documented Stripe API credential.
`STRIPE_PUBLISHABLE_KEY`	Documented Stripe publishable key.
`PAYMENT_CURRENCY`	Passed to Payify configuration.
`INTEGROMAT_BASE_URI`	Integromat webhook base URI.
`INTEGROMAT_BOOKING_HOOK`	Booking webhook configuration.

`config/initializers/payify.rb` configures the currency. `config/initializers/integromat.rb` reads the webhook variables and configures the webhook integration.

Configuration detail to verify: the README's Docker instructions mention `DB_HOST=db`, whereas `config/database.yml` reads `DATABASE_HOST`. The source therefore contains a configuration-name discrepancy worth documenting before attempting a local run.

The README also instructs users to run database creation, migrations, and seeds. A valid local database and any required integration credentials must be available for the corresponding features to work. Successful startup has not been verified here.

11. Important architectural components

This section documents the observed organization and responsibilities without assigning design-pattern labels.

Request routing and controller layer

`config/routes.rb`, `ApplicationController`, resource controllers, and shared controller concerns.

Data and domain logic

Active Record models, associations, validations, model callbacks, and database migrations.

Authentication and authorization

Devise configuration, Pundit integration, and application policy classes.

Presentation and client-side behavior

ERB views, view partials, JavaScript controllers, SCSS, and locale translations.

External integrations

Payify payment functionality and Integromat webhook configuration.

Testing and execution infrastructure

RSpec configuration, factories, Docker files, database setup, and Rails command-line scripts.

Important cross-module interactions

1. `ApplicationController` includes the shared concerns used by the resource controllers.
2. `RoomsController`, `BookingsController`, and `SettingsController` include `CrudConcern`.
3. `Booking` includes Payify's payment concern and the application's ownership and pricing concerns.
4. `Booking` callbacks handle payment-record creation and notification-related behavior.
5. `User` includes settings-related functionality.
6. Controllers invoke Pundit authorization, while the policy classes define permissions.
7. Routes connect browser requests to controllers and mount the Payify engine.

These are source-observed relationships. The internal implementation of third-party gems and any production-only behavior are outside the evidence available in this source snapshot.

12. Files relevant to later source analysis

The following files should receive closer examination because they contain shared logic, cross-module interactions, validation, callbacks, or integration boundaries.

Priority	File(s)	Why to inspect closely
High	`app/controllers/concerns/crud_concern.rb`	Shared request-handling methods used by multiple controllers.
High	`app/controllers/rooms_controller.rb`	Room-specific configuration and booking-form initialization.
High	`app/controllers/bookings_controller.rb`	Booking operations, cancellation, and payment-intent initialization.
High	`app/models/booking.rb`	Validations, callbacks, pricing/payment methods, and notifications.
High	`app/models/concerns/has_price_concern.rb`	Shared price-calculation and persistence behavior.
High	`app/models/concerns/has_owner_concern.rb`	Owner assignment and current-user interaction.
High	`app/models/concerns/has_settings_concern.rb`	Settings association and user-creation callback.
High	`app/controllers/application_controller.rb`	Common controller setup and included modules.
Medium	`app/models/room.rb`	Room data, associations, and availability-related queries.
Medium	`app/models/user.rb`	Authentication, associations, and current-user lookup.
Medium	`app/controllers/settings_controller.rb`	Settings operations and record retrieval.
Medium	`app/controllers/payify/payments_controller.rb`	Payment completion handling.
Medium	`app/policies/*.rb`	Authorization rules and ownership checks.
Medium	`config/routes.rb`	Mapping of application requests and external engine mounting.
Medium	`config/initializers/payify.rb`	Payment integration configuration.
Medium	`config/initializers/integromat.rb`	Webhook configuration.

Medium	`spec/models/booking_spec.rb`	Existing evidence for booking and pricing behavior.
Medium	`spec/models/room_spec.rb`	Existing evidence for room creation.
Medium	`db/schema.rb` and `db/migrate/`	Persistent data structure and schema history.

Source Inspection Map

Use this as the practical checklist for Part 1.

Inspection checklist

0 of 8

- ◯ **Application entry and routing**
  Trace how requests enter the application and receive shared setup.
  config/routes.rb; config/application.rb; app/controllers/application_controller.rb

- ◯ **Shared controller behavior**
  Record shared methods and determine which controllers use them.
  app/controllers/concerns/crud_concern.rb; app/controllers/concerns/auth_concern.rb; app/controllers/concerns/locale_concern.rb; app/controllers/concerns/page_concern.rb

- ◯ **Core resource operations**
  Trace resource-specific actions, parameters, authorization, and redirects.
  app/controllers/rooms_controller.rb; app/controllers/bookings_controller.rb; app/controllers/settings_controller.rb

- ◯ **Model behavior**
  Document associations, validations, callbacks, and model methods.
  app/models/room.rb; app/models/booking.rb; app/models/user.rb; app/models/setting.rb

- ◯ **Cross-module model logic**
  Follow behavior shared by multiple models.
  app/models/concerns/has_owner_concern.rb; app/models/concerns/has_price_concern.rb; app/models/concerns/has_settings_concern.rb

- ◯ **Authorization**
  Verify the actual permission rules and their callers.
  app/policies/application_policy.rb; app/policies/room_policy.rb; app/policies/booking_policy.rb; app/policies/setting_policy.rb

Integrations and configuration

Trace integration entry points and environment dependencies.

app/controllers/payify/payments_controller.rb; config/initializers/payify.rb; config/initializers/integromat.rb; config/database.yml

Persistence and tests

Verify data relationships, migration history, and existing test evidence.

db/schema.rb; db/migrate/; spec/models/booking_spec.rb; spec/models/room_spec.rb; spec/rails_helper.rb

Important limitations to record in your report

- The exact commit corresponding to the supplied GitHub URL is **not confirmed from source**.
- The current state of the URL could not be inspected because it returned 404.
- The source snapshot's test suite has not been executed during this inspection.
- Third-party gem internals and production environment behavior are not established by the application source alone.
- No design patterns have been identified or classified in this report.

This is a source inventory and architecture overview only. The course instructions also require recording the source URL, source commits, environment versions, run/test commands, inspected modules, and limitations in the project README. DP_Csestudy.pdf DP_Csestudy.pdf

Next step: Before treating this as the definitive baseline for your submission, provide the exact source ZIP associated with the supplied GitHub repository, or restore access to that repository. That will let us verify the snapshot and document the exact source version without guessing.

Now begin Part 1 of my Design Pattern Case Study.

The official assignment instructions require patterns.csv to contain confirmed pattern collaborations, including all applicable participants, their actual source locations, and evidence from the code.

Analyze the uploaded application source code systematically for design patterns.

Consider relevant categories including:
- Creational
- Structural
- Behavioral

- GRASP
- Any other pattern category that is actually applicable

IMPORTANT RULES:

1. Do NOT assume a pattern exists merely because the code resembles a textbook example.
2. A pattern must be supported by an actual collaboration in the source code.
3. Do not invent classes, interfaces, functions, or relationships.
4. If a textbook role is combined into another class, explicitly document that.
5. Include ALL applicable participants, not just the main class.
6. Use the exact class/interface/function names from the source.
7. Give exact source file paths and line ranges.
8. Explain the concrete interaction between participants.
9. Explain what problem the collaboration solves.
10. Distinguish CONFIRMED patterns from merely POSSIBLE/CANDIDATE patterns.
11. If you cannot verify something from the source, mark it as "cannot confirm".
12. Do not count the same collaboration multiple times simply because multiple classes participate in it.

For every CONFIRMED pattern, produce:

- pattern_name
- pattern_type
- instance_name
- participant_roles
- source_locations
- intent_and_evidence

Before producing the final list, inspect the relevant source files carefully.

Do NOT create the final CSV yet.

First give me a detailed evidence-based pattern inventory so I can review it.

Worked for 54s

Part 1 — Evidence-Based Design Pattern Inventory

1. Source baseline and evidence rules

The inspected baseline is the Rails application under `A1/data/code/original/` in the archived `A1_submission_ready.zip`.

I inspected the shared CRUD concern, resource controllers, models, model concerns, policies, routes, relevant ERB views, initializers, and existing model specifications. The important distinction for this analysis is between:

- **Confirmed:** the actual source contains the collaboration needed to justify the pattern, with concrete participants and interactions.
- **Candidate:** the code exhibits some relevant characteristics, but the evidence does not establish the complete pattern or its defining collaboration.
- **Not confirmed:** a pattern may be suggested by a name, callback, dependency, or framework convention, but the required implementation or interaction cannot be verified.

I will also distinguish application-written code from behavior supplied by Rails or third-party gems. For example, using a gem that supports a pattern does not automatically establish that the application implements that pattern itself.

Source-path convention: all paths below are relative to `A1/data/code/original/` unless otherwise stated. Line numbers refer to the inspected archived files. They must be checked against the exact repository commit before being used in the final evidence CSV.

2. Confirmed pattern inventory

I found five strong candidates for inclusion as confirmed pattern instances. The first two are framework-level architecture/data-access patterns; the next three concern concrete application collaborations.

A distinction matters here: the fact that a class has a particular name or uses inheritance is not sufficient. Each finding below is tied to actual code behavior.

P1. Model–View–Controller (MVC)

Confirmed

Category: Architectural pattern

Instance name: Room listing and booking request flow

Problem solved: Separates HTTP request handling, accommodation/booking data and business rules, and presentation of the resulting page. The observed application uses Rails' controllers, Active Record models, and ERB templates for these responsibilities.

Participants and evidence

Role	Actual participant	Source location
Request routing	`Rails.application.routes.draw`	`config/routes.rb:3–26`
Controller	`RoomsController`	`app/controllers/rooms_controller.rb:1–41`
Controller	`BookingsController`	`app/controllers/bookings_controller.rb:1–47`
Model	`Room`	`app/models/room.rb:1–40`
Model	`Booking`	`app/models/booking.rb:1–72`
View	Room listing template	`app/views/rooms/index.html.erb:1–21`
View	Booking details template	`app/views/bookings/show.html.erb:1–91`
Shared controller base	`ApplicationController`	`app/controllers/application_controller.rb:1–14`

Concrete interaction

1. `config/routes.rb` maps room and booking URLs to their respective resource controllers.
2. `RoomsController` includes `CrudConcern` and supplies room-specific behavior.
3. `Room` supplies the model data and methods used to render accommodation information.
4. The room index view renders the collection in `@objects`.
5. Booking requests are handled by `BookingsController`, which operates on `Booking` records and prepares them for the corresponding views.

For direct evidence, `app/views/rooms/index.html.erb:13` renders `@objects`, while `app/views/bookings/show.html.erb:10–11` reads the room title and location through the booking's room association.

Assessment: Confirmed. This is an actual application-level MVC collaboration, not merely an inference from the presence of Rails.

CSV note: This should be recorded as one architectural collaboration only if your team treats the connected room/booking flow as one instance. Do not create a separate MVC record for every class or template simply because they participate in the same overall architecture.

P2. Active Record

Confirmed

Category: Data-access / architectural pattern

Instance name: Booking and room persistence

Problem solved: Represents application data as model objects and provides database querying, persistence, associations, and validations without requiring the application to implement all SQL and mapping operations manually.

Participants and evidence

Role	Actual participant	Source location
Base persistent-model class	`ApplicationRecord`	`app/models/application_record.rb:1–3`
Persistent model	`Booking`	`app/models/booking.rb:1–15`
Persistent model	`Room`	`app/models/room.rb:1–15`
Database schema	`bookings` and `rooms` tables	`db/schema.rb:45–58`, `74–104`
Database schema	`users` table	`db/schema.rb:114–125`

Concrete interaction

`ApplicationRecord` inherits from `ActiveRecord::Base` and declares itself the primary abstract class. `Booking` and `Room` inherit from `ApplicationRecord`, declare associations, and perform model-level validation.

For example:

- `Booking` declares `belongs_to :user` and `belongs_to :room` at `app/models/booking.rb:10–11`.
- `Room` declares `belongs_to :user` and `has_many :bookings` at `app/models/room.rb:7–9`.
- `Booking#dates_are_available` queries the associated room's bookings at `app/models/booking.rb:21–25`.

The database schema contains corresponding `bookings`, `rooms`, and `users` tables.

Assessment: Confirmed. The inheritance, model operations, and corresponding persistence schema are directly visible.

Important qualification: This is a framework-provided Active Record implementation used by the application. It should not be described as a custom ORM developed by the project.

P3. Template Method — shared CRUD workflow

Confirmed, with qualification

Category: Behavioral

Instance name: Shared resource CRUD processing

Problem solved: Reuses common create, read, update, delete, authorization, validation, pagination, and redirect handling while allowing each resource controller to provide its own model-specific operations.

This is the strongest application-level behavioral pattern candidate in the inspected source. The implementation uses a shared Ruby module and method overrides, rather than a textbook abstract superclass.

Participants and evidence

Role	Actual participant	Source location
Shared algorithm and default behavior	`CrudConcern`	`app/controllers/concerns/crud_concern.rb:1–149`
Shared create algorithm	`CrudConcern#create`	`app/controllers/concerns/crud_concern.rb:28–40`
Shared update algorithm	`CrudConcern#update`	`app/controllers/concerns/crud_concern.rb:48–60`
Shared lookup operation	`CrudConcern#set_object`	`app/controllers/concerns/crud_concern.rb:142–149`
Concrete resource participant	`RoomsController`	`app/controllers/rooms_controller.rb:1–34`
Concrete resource participant	`BookingsController`	`app/controllers/bookings_controller.rb:1–40`
Concrete resource participant	`SettingsController`	`app/controllers/settings_controller.rb:1–34`

Concrete interaction

`RoomsController`, `BookingsController`, and `SettingsController` each include `CrudConcern`. The concern implements shared actions, but delegates important resource-specific details to methods such as `base_class`, `object_class`, `model_params`, and `strong_params`.

The following examples establish the interaction:

- `CrudConcern#create` invokes `object_class.new(form_values)`, validates the record, saves it, and redirects on success (`crud_concern.rb:28–40`).
- `RoomsController#base_class` returns `Room` (`rooms_controller.rb:8–10`).
- `BookingsController#base_class` returns `Booking` (`bookings_controller.rb:18–20`).
- `SettingsController#base_class` returns `Setting` (`settings_controller.rb:6–8`).
- `SettingsController` overrides `set_object` to retrieve the current user's setting (`settings_controller.rb:30–34`) and overrides `redirect_to_show` to redirect to the settings page (`settings_controller.rb:36–38`).

This means that a shared workflow invokes resource-specific methods without having to duplicate the entire CRUD implementation in every controller.

Assessment: Confirmed as a module-based Template Method implementation, with a qualification about its Ruby structure. The source demonstrates an actual shared algorithm and resource-specific hook methods. It is not an exact textbook superclass implementation, so the report should state that difference rather than conceal it.

Counting rule: This is one shared CRUD collaboration across the three controllers, not three separate Template Method instances merely because three controllers include the concern.

P4. GRASP Controller

Confirmed

Category: GRASP

Instance name: Booking request handling

Problem solved: Assigns responsibility for handling incoming booking-related system operations to a controller rather than placing the entire HTTP request-handling workflow inside a domain model.

Participants and evidence

GRASP role	Actual participant	Source location
Controller for booking requests	`BookingsController`	`app/controllers/bookings_controller.rb:1-47`
Common request-handling base	`ApplicationController`	`app/controllers/application_controller.rb:1-14`
Domain object	`Booking`	`app/models/booking.rb:1-72`
Routing entry point	Booking routes	`config/routes.rb:17-21`

Concrete interaction

`config/routes.rb:17-21` declares booking resources and a cancellation action. `BookingsController#cancel` then performs the request-level sequence:

1. Calls `set_object` to retrieve the booking.
2. Calls `authorize @object`.
3. Calls `@object.cancel!`.
4. Redirects to the booking page.

The sequence is visible at `app/controllers/bookings_controller.rb:6-14`. The domain operation itself is delegated to `Booking#cancel!` at `app/models/booking.rb:59-61`.

This division demonstrates that the controller handles the incoming system event and coordinates the corresponding domain operation.

Assessment: Confirmed for the booking-request collaboration.

Counting rule: Do not record `BookingsController`, its cancellation action, and its show action as multiple instances of the same GRASP Controller collaboration without identifying a genuinely distinct responsibility or system event. If the CSV records one instance per system operation, use that granularity consistently for the other controllers too.

P5. GRASP Creator

Confirmed

Category: GRASP

Instance name: Initialize a booking for a room

Problem solved: Assigns the responsibility for constructing a booking through the room's existing association, since a room has bookings and the association provides a direct way to create one linked to

that room.

Participants and evidence

GRASP role	Actual participant	Source location
Client initiating creation	`RoomsController#init_booking_form`	`app/controllers/rooms_controller.rb:36-40`
Associated object through which creation occurs	`Room#bookings` association	`app/models/room.rb:7-9`
Created domain object	`Booking`	`app/models/booking.rb:1-15`
Association endpoint	`Booking#room`	`app/models/booking.rb:10-11`

Concrete interaction

The controller's `init_booking_form` method first checks whether the user is signed in. It then executes:

```ruby
@booking = @object.bookings.new num_guests: 1, user_id: current_user.id
```

Source: `app/controllers/rooms_controller.rb:36-40`.

The association comes from `Room` declaring `has_many :bookings` at `app/models/room.rb:8`. `Booking` declares `belongs_to :room` at `app/models/booking.rb:11`.

The code therefore creates a booking through the selected room's collection and initializes the guest count and user ID for the form.

Assessment: Confirmed. This is a concrete GRASP Creator collaboration, with the room association used as the creation path. The constructor itself is supplied by Active Record's association API, not by a custom `Room#create_booking` method.

Counting rule: This is one booking-initialization collaboration. Do not count the same `@object.bookings.new` expression as a separate instance for every role participating in it.

3. Possible or candidate patterns — not confirmed

These findings should remain separate from the confirmed inventory. I would not add them to `patterns.csv` as confirmed instances without stronger evidence.

C1. GRASP Information Expert

Candidate

Potential instance: Booking date-availability validation

Relevant source

- `app/models/booking.rb:10–25`
- `app/models/room.rb:7–9, 35–39`

Evidence: `Booking#dates_are_available` uses the booking's date fields and queries `room.bookings` to find conflicting reservations. `Room#booked_dates` separately retrieves booked date ranges.

Why it is a candidate: The code places relevant date-validation responsibility in `Booking` and date-list retrieval in `Room`. This is consistent with assigning responsibilities to objects with relevant information.

Why I would not yet confirm it as a separate pattern instance: Information Expert is a responsibility-assignment principle, not simply the fact that a model contains a method. We would need to define the exact responsibility and show why the selected participant has the information needed to fulfil it. The current evidence supports a plausible application, but I would not count it separately from the booking collaboration until that analysis is agreed.

C2. Observer

Candidate

Potential instance: Booking lifecycle and external notification

Relevant source

- `app/models/booking.rb:7–8`
- `app/models/booking.rb:67–71`

Evidence: `Booking` registers `after_save :create_payment` and `after_create :notify_booking`. The notification method checks whether a booking webhook is configured and, if so, invokes `Integromat::Webhook.new(:booking).trigger(...)`.

Why it resembles Observer: The application reacts to lifecycle events, and a booking creation can trigger an external notification.

Why it is not confirmed: Rails model callbacks do not, by themselves, prove a GoF Observer collaboration. The source does not show a subject maintaining a collection of observers, an observer interface, or multiple registered observer objects being notified. The Integromat webhook is an external integration, not a confirmed observer participant.

Decision: Do not record Observer as confirmed on the current evidence.

C3. Strategy

Candidate

Potential instance: Alternative booking-pricing or payment behavior

Relevant source

- `app/models/booking.rb:2-5, 63-65`
- `app/models/concerns/has_price_concern.rb:8-26`
- `config/initializers/payify.rb:1-3`

Evidence: Pricing is implemented in `HasPriceConcern`, while payment functionality is included through `Payify::HasPaymentConcern` and configured through Payify.

Why it resembles Strategy: Pricing and payment are distinct responsibilities that could, in principle, have alternative implementations.

Why it is not confirmed: The application source does not establish a family of interchangeable algorithms selected through a common strategy interface. There is no demonstrated runtime selection among alternative pricing strategies.

Decision: Do not record Strategy as confirmed. Separate responsibilities and third-party integration are not enough.

C4. Factory / Factory Method

Candidate

Potential instance: Creation of bookings and other records

Relevant source

- `app/controllers/concerns/crud_concern.rb:22-40`
- `app/controllers/rooms_controller.rb:36-40`
- `spec/factories/rooms.rb:1-28`
- `spec/factories/bookings.rb:1-5`
- `spec/factories/users.rb:1-8`

Evidence: The application constructs records using `object_class.new(...)` and `@object.bookings.new(...)`. Test specifications use Factory Bot definitions to build test data.

Why it resembles a factory: Object construction occurs through framework-provided methods and through named test-data factory definitions.

Why it is not confirmed: The production code does not establish a dedicated factory participant encapsulating construction decisions or a Factory Method collaboration. Factory Bot is used for test-data setup, which is not evidence of a production object-creation pattern.

Decision: Do not record Factory Method or Abstract Factory as confirmed.

C5. Singleton

Candidate

Potential instance: Current-user scope

Relevant source

- `config/initializers/current_scope.rb:1-3`
- `app/controllers/concerns/auth_concern.rb:1-6`
- `app/models/user.rb:14-16`

Evidence: `CurrentScope` is a module exposing a thread-local `user_id`. `AuthConcern#set_current_scope` assigns the current user's ID, and `User.current` retrieves the corresponding user.

Why it resembles shared state: The module provides a shared access point to thread-local context.

Why it is not confirmed: The source does not show a class with a controlled single-instance construction mechanism. A module-level accessor and thread-local storage do not establish the GoF Singleton pattern.

Decision: Do not record Singleton as confirmed.

C6. Facade

Candidate

Potential instance: Shared CRUD operations

Relevant source: `app/controllers/concerns/crud_concern.rb:8–149`

Evidence: The concern exposes common resource operations and handles several details internally, including pagination, parameter processing, authorization, validation, persistence, and redirects.

Why it resembles Facade: It provides a common set of operations to controllers.

Why it is not confirmed: The source does not demonstrate a dedicated facade providing a simplified interface to a distinct, complex subsystem. The concern is directly implementing shared controller behavior, which is better supported by the confirmed shared CRUD collaboration described earlier.

Decision: Do not record Facade as confirmed, and do not count the same CRUD collaboration again under this name.

4. Other source structures examined

These structures are real and relevant to the application, but their existence alone does not justify an additional confirmed design-pattern record.

Structure	Exact source	Evidence-based conclusion
Shared Ruby concerns	`app/controllers/concerns/*.rb`; `app/models/concerns/*.rb`	Confirmed module reuse. Do not automatically classify each mixin as a separate GoF pattern.
Pundit authorization	`app/controllers/application_controller.rb:4`; `app/policies/*.rb`	Confirmed authorization integration. A distinct Proxy or Strategy implementation is not established by this alone.
Payment integration	`app/models/booking.rb:2, 7`; `app/controllers/payify/payments_controller.rb:1–12`	Confirmed use of Payify interfaces. The gem's internal pattern collaborations cannot be established from the application's code.

Webhook integration	`app/models/booking.rb:67–71` ; `config/initializers/integromat.rb:1–9`	Confirmed external notification call. No confirmed Observer or Adapter collaboration.
Layout and partial rendering	`app/views/layouts/base.html.erb:4–14` ; `app/views/rooms/index.html.erb:13`	Confirmed reuse of ERB partials and layouts. This does not, by itself, establish a GoF Composite or Decorator pattern.
Test data factories	`spec/factories/*.rb`	Confirmed testing utility usage. Exclude from production pattern counts unless the assignment explicitly includes test infrastructure as a separate source version or scope.

5. Important unresolved source questions

These are evidence gaps, not proof that a pattern exists.

1. `create_setting` : `app/models/concerns/has_settings_concern.rb:4–8` registers `after_create :create_setting` , but the implementation of `create_setting` is not defined in the inspected application files. Its runtime implementation and any associated creation collaboration cannot be confirmed from this source snapshot.

2. `create_payment` : `app/models/booking.rb:7` registers the callback, and `Booking` includes `Payify::HasPaymentConcern` at line 2. The implementation of `create_payment` is not present in the application source. The callback's third-party behavior cannot be fully verified here.

3. **Payify payment completion:** `app/controllers/payify/payments_controller.rb:3–10` includes `Payify::PaymentsControllerConcern` and maps the payment model to `@booking` . The full completion flow is partly supplied by the external gem and cannot be reconstructed from this file alone.

4. **Exact baseline commit:** The archived application is available, but the exact commit associated with the inaccessible GitHub URL remains unverified. The source line ranges above must be reconciled with the actual submission baseline before final CSV generation.

6. Consolidated inventory for review

This is the proposed review list, not the final CSV.

ID	Pattern	Category	Status	Proposed instance
P1	Model–View–Controller	Architectural	Confirmed	Room listing and booking request flow

P2	Active Record	Data-access / architectural	Confirmed	Booking and room persistence
P3	Template Method	Behavioral	Confirmed, qualified	Shared resource CRUD processing
P4	GRASP Controller	GRASP	Confirmed	Booking request handling
P5	GRASP Creator	GRASP	Confirmed	Initialize a booking for a room
C1	GRASP Information Expert	GRASP	Candidate	Booking date-availability validation
C2	Observer	Behavioral	Candidate	Booking lifecycle and external notification
C3	Strategy	Behavioral	Candidate	Alternative booking-pricing or payment behavior
C4	Factory Method / Factory	Creational	Candidate	Record creation
C5	Singleton	Creational	Candidate	Current-user scope
C6	Facade	Structural	Candidate	Shared CRUD operations

The five confirmed findings are distinct at the level of their documented responsibilities, although P1 and P2 are architectural/framework-level findings rather than individual GoF class collaborations. Before producing the CSV, the team should decide whether the assignment expects those broad patterns to be counted alongside instance-level collaborations.

My recommendation for the next review step: validate P1–P5 against the exact baseline commit and agree on the counting granularity for MVC, Active Record, and GRASP Controller. Keep C1–C6 out of the confirmed totals unless further source evidence establishes their defining collaborations. No CSV has been created, and no source files have been modified.

Now independently verify every pattern you identified in Part 1 against the actual uploaded source code.

For EACH proposed pattern:

1. Re-open/inspect the relevant source files.
2. Verify that every participant actually exists.
3. Verify that every stated relationship actually exists.
4. Verify the source file paths.

5. Verify the line ranges.
6. Verify that the interaction described actually occurs.
7. Verify that the pattern solves the stated problem.
8. Check whether this is genuinely a design-pattern collaboration or merely similar-looking code.
9. Check for duplicate instances.
10. Check whether multiple concrete classes belong to one collaboration and therefore should count as ONE instance according to the assignment rules.

Create a verification table:

Pattern | Instance | Confirmed? | Participants Verified? | Locations Verified? | Evidence Verified? | Corrections

Be conservative.

If a previously identified pattern cannot be confirmed, remove it from the confirmed list rather than forcing it to fit.

Do not invent evidence.

Worked for 35s

1. Verification results against the archived source

I reopened the relevant Ruby files and inspected their numbered source lines, including the existing `A1/Part1/patterns.csv`. This also revealed that the earlier inventory was incomplete: the existing Part 1 records already include two GRASP Information Expert collaborations that deserve separate verification.

One important correction: **the earlier answer treated some framework conventions as confirmed patterns too readily.** The revised inventory below distinguishes the patterns supported by application code from those that should not be included as confirmed records.

The source inspected is the archived `A1/data/code/original/` snapshot. This verifies that snapshot, but not an independently fetched current GitHub revision.

2. Verification table

Pattern	Instance	Confirmed?	Participants verified?	Locations verified?	Evidence verified?	Corrections
MVC	Room listing and booking flow	**Yes**	Yes	Yes	Yes	Keep as one architectural instance; do not count each controller/view separately.
Active Record	Booking and room persistence	**Yes, framework-level**	Yes	Yes	Yes	State that Rails provides the implementation; this is not a custom ORM.
Template Method	Shared resource CRUD workflow	**Yes, qualified**	Yes	Yes	Yes	One shared collaboration across three controllers; module-based implementation, not superclass inheritance.
GRASP Controller	Booking cancellation	**Yes**	Yes	Yes	Yes	Use the specific `BookingsController#cancel` collaboration, not a second overlapping booking-request record.
GRASP Creator	Initialize a booking for a room	**Qualified; retain with corrected roles**	Yes	Yes	Yes	`RoomsController#init_booking_form` calls the room association's `new`; `Room` is the associated creator context. No custom room factory method exists.
GRASP Information Expert	Booking date-availability validation	**Yes**	Yes	Yes	Yes	Promote from candidate to confirmed: `Booking` has the dates and queries `room.bookings`.
GRASP Information Expert	Booking price calculation	**Yes**	Yes	Yes	Yes	Separate responsibility from date availability; `Booking` includes `HasPriceConcern`, which obtains price and VAT data from associated objects.
Observer	Booking lifecycle notification	**No**	Partial	Yes	Partial	Callbacks and a webhook do not establish an Observer

						collaboration. Remove from confirmed records.
Strategy	Alternative pricing/payment behavior	**No**	No strategy participants established	Yes	No	No interchangeable algorithm family or strategy-selection collaboration was found.
Factory Method / Factory	Production record creation	**No**	Partial	Yes	No	`object_class.new` and association creation do not establish a dedicated factory pattern. Factory Bot is test infrastructure.
Singleton	Current-user scope	**No**	Partial	Yes	No	`CurrentScope` uses thread-local module state; no single-instance object lifecycle is demonstrated.
Facade	Shared CRUD operations	**No**	Yes	Yes	No	`CrudConcern` implements shared controller behavior; the source does not establish a facade over a separate subsystem.

Interpretation: "Locations verified" means the listed file and line ranges exist in the inspected snapshot. It does not mean every pattern claim is valid; the evidence column separately evaluates whether the actual interactions justify the pattern.

3. Detailed evidence corrections

3.1 MVC — retain

Verified files and lines

- `config/routes.rb:3–26`
- `app/controllers/rooms_controller.rb:1–41`
- `app/controllers/bookings_controller.rb:1–47`
- `app/models/room.rb:1–40`
- `app/models/booking.rb:1–72`
- `app/views/rooms/index.html.erb:1–23`
- `app/views/bookings/show.html.erb:1–91`

The routes map requests to controllers. The controllers work with models, and the views render model-backed data. For example, `RoomsController` includes `CrudConcern` at line 2, while the room index view renders `@objects` at line 13.

Decision: Retain MVC as one architectural collaboration. Do not create separate instances for every model/view pair in the same request flow.

3.2 Active Record — retain with scope qualification

Verified files and lines

- `app/models/application_record.rb:1–3`
- `app/models/booking.rb:1–15`
- `app/models/room.rb:1–15`
- `db/schema.rb` — the relevant table definitions

`ApplicationRecord` inherits from `ActiveRecord::Base` and calls `primary_abstract_class`. `Booking` and `Room` inherit from `ApplicationRecord`, declare associations, and use Active Record queries and validations.

For example, `Booking#dates_are_available` queries `room.bookings` at `app/models/booking.rb:21–25`.

Decision: Retain as an Active Record pattern instance if the assignment accepts framework/data-access patterns. Record it as one persistence collaboration, not one instance per model. Clearly identify the implementation as Rails-provided.

3.3 Template Method — retain, but map participants precisely

Verified files and lines

- `app/controllers/concerns/crud_concern.rb:8–68`
- `app/controllers/concerns/crud_concern.rb:101–149`
- `app/controllers/rooms_controller.rb:1–34`
- `app/controllers/bookings_controller.rb:1–40`
- `app/controllers/settings_controller.rb:1–38`

`CrudConcern` supplies shared CRUD operations. The concrete controllers include the same concern and provide resource-specific behavior through methods such as `base_class`, `object_class`, and `model_params`.

The strongest evidence is `CrudConcern#create` at lines 28–40 and the implementations of `base_class` and `object_class` in the three controllers.

Decision: Retain one instance. In the participant roles, explicitly state that the template is a Ruby module mixed into the concrete controllers. Do not invent an abstract superclass or claim that the concrete classes inherit from `CrudConcern`.

3.4 GRASP Controller — retain one specific instance

Verified files and lines

- `config/routes.rb:17–21`
- `app/controllers/bookings_controller.rb:6–14`
- `app/models/booking.rb:59–61`

The cancellation route reaches `BookingsController#cancel`. That action retrieves and authorizes the booking, invokes `Booking#cancel!`, and redirects to the booking page. `Booking#cancel!` performs the state update.

Decision: Retain one `Booking Cancellation Request Handling` instance. The earlier broader wording, "Booking request handling," should be narrowed to this specific interaction to prevent overlap with the cancellation instance.

3.5 GRASP Creator — retain only with corrected role mapping

Verified files and lines

- `app/controllers/rooms_controller.rb:36–40`
- `app/models/room.rb:7–9`
- `app/models/booking.rb:10–11`

The source explicitly executes:

```ruby
@booking = @object.bookings.new num_guests: 1, user_id: current_user.id
```

The `Room` model declares `has_many :bookings`, and `Booking` declares `belongs_to :room`.

This is consistent with the GRASP Creator guideline because a room contains or aggregates its bookings. However, the code does not define a custom `Room#create_booking` method.

Decision: Retain only if the assignment accepts framework association-based creation as evidence for GRASP Creator. Map `Room` as the creator context through its association, `Booking` as the created object, and `RoomsController#init_booking_form` as the initiating client. Do not label the controller itself as the GRASP Creator.

3.6 GRASP Information Expert — date availability

Verified files and lines

- `app/models/booking.rb:10–15`
- `app/models/booking.rb:21–25`
- `app/models/room.rb:7–8`

`Booking` contains the booking dates and validates their availability by querying the associated room's bookings, excluding the current record and checking for overlapping intervals.

Decision: Promote to confirmed. The code provides concrete evidence of responsibility being assigned to an object that has access to the relevant information. This is one distinct date-availability instance.

3.7 GRASP Information Expert — price calculation

Verified files and lines

- `app/models/booking.rb:1–5`
- `app/models/concerns/has_price_concern.rb:4–26`
- `app/models/room.rb:7–8`
- `app/models/user.rb:1–8`
- `app/models/setting.rb:1–3`

`Booking` includes `HasPriceConcern` . The concern calculates the pre-tax total from `room.day_price` and the booking duration, obtains VAT through `room.user.setting.vat` , and calculates the tax-inclusive total. It registers callbacks to update prices when booking dates change.

Decision: Confirm as a second GRASP Information Expert instance, distinct from date availability. The source supports this responsibility, but the participant roles must include the concern and the associated objects that supply the information.

4. Why the remaining candidates are excluded

Candidate	Source evidence	Why it fails the confirmation test

Observer	`app/models/booking.rb:7–8, 67–71`	The callbacks invoke payment and notification behavior, but there is no demonstrated subject–observer registration and notification collaboration.
Strategy	`app/models/concerns/has_price_concern.rb:8–26` ; `config/initializers/payify.rb:1–3`	There is no verified set of interchangeable pricing algorithms or common strategy interface.
Factory Method / Factory	`app/controllers/concerns/crud_concern.rb:28–40` ; `app/controllers/rooms_controller.rb:36–40`	These are ordinary Active Record object-construction calls. The code does not establish a dedicated production factory collaboration.
Singleton	`config/initializers/current_scope.rb:1–3` ; `app/models/user.rb:14–16`	Thread-local module state is present, but a Singleton instance lifecycle is not demonstrated.
Facade	`app/controllers/concerns/crud_concern.rb:8–149`	The concern provides shared controller operations, but a separate subsystem and a simplified interface over it are not demonstrated.

These candidates should not appear as confirmed rows in the final `patterns.csv` . They can be discussed in explanatory notes if your report requires rejected candidates.

5. Duplicate-instance audit

The assignment says that multiple concrete classes in one collaboration must not inflate the count. Applying that rule:

Collaboration	Number of instances to count	Reason
Shared CRUD / Template Method	1	`RoomsController` , `BookingsController` , and `SettingsController` share the same `CrudConcern` workflow.
MVC	1	The selected room/booking request flow is one architectural example; individual templates and controllers are participants, not additional instances.
Active Record	1	The `Room` and `Booking` models participate in the same framework-based persistence approach.
Booking cancellation / GRASP Controller	1	One controller coordinates one defined cancellation operation.

Room-to-booking creation / GRASP Creator	1	One booking-initialization collaboration through the room association.
Booking date availability / GRASP Information Expert	1	One date-availability responsibility.
Booking price calculation / GRASP Information Expert	1	One separate price-calculation responsibility.

Do not count the same shared CRUD workflow again as Facade. Likewise, do not count the same booking callback once as Observer and again as a separate callback pattern.

6. Revised recommendation for Part 1

The source supports **six distinct pattern names**, representing seven proposed instances if both Information Expert responsibilities are retained:

1. Model–View–Controller — 1 instance.
2. Active Record — 1 instance.
3. Template Method — 1 instance.
4. GRASP Controller — 1 instance.
5. GRASP Creator — 1 instance, subject to the association-based role qualification above.
6. GRASP Information Expert — 2 instances: date availability and price calculation.

This is a proposed inventory for review, not a final count to submit. MVC and Active Record are framework/architectural patterns, while Template Method and the GRASP entries have distinct responsibility-assignment or behavioral evidence. Confirm that your instructor accepts these categories and the chosen counting granularity before finalizing the dataset.

Verification limitation: These line numbers and interactions have been checked against the accessible archived source snapshot. The exact current GitHub revision remains unverified, so do not represent this as verification against a fresh repository checkout. No CSV has been created or source code modified.

Now create the final Part 1 patterns.csv content based ONLY on the patterns that were confirmed during verification.

Use exactly these columns from the assignment:

source_version
pattern_name
pattern_type
instance_name

participant_roles
source_locations
intent_and_evidence

Requirements:

- One row = one confirmed pattern collaboration in one source version.
- Include all applicable participants.
- Use actual class/interface/function names.
- Use exact source paths and line ranges.
- Do not invent roles.
- If roles are combined into one class, state that explicitly.
- Do not include unconfirmed candidate patterns.
- Keep the same instance together rather than splitting it into multiple fake instances.
- Quote CSV fields when necessary.
- Ensure valid UTF-8 CSV formatting.

Use source_version = "original".

First briefly state the final number of confirmed pattern instances.

Then output ONLY the complete CSV inside a code block so I can save it as:

Part1/patterns.csv

Worked for 14s

Final number of confirmed pattern instances: **7**.

CSV

```csv
source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evidence
"original","Model-View-Controller","Architectural","Room Listing and Booking Request Flow","Routing: Rails.app
"original","Active Record","Data Access","Room and Booking Persistence","Abstract base model: ApplicationRecor
"original","Template Method","Behavioral","Shared Resource CRUD Workflow","Shared workflow and hook definition
"original","GRASP Controller","GRASP","Booking Cancellation Request Handling","System-operation controller: Bo
"original","GRASP Creator","GRASP","Initialize a Booking for a Room","Initiating client: RoomsController#init_
```

"original","GRASP Information Expert","GRASP","Booking Date Availability Validation","Information expert: Book
"original","GRASP Information Expert","GRASP","Booking Price Calculation","Host object: Booking; Responsibilit

Using ONLY the confirmed Part 1 patterns.csv records we just established, create the corresponding pattern_counts.csv.

Follow the assignment exactly.

Columns:

source_version
summary
pattern_name
count

For per_pattern rows:
- summary = per_pattern
- pattern_name = exact pattern name
- count = number of confirmed instances

Also provide the required total rows:
- total_pattern_types
- total_pattern_instances

Important:
- Count from the confirmed pattern records.
- Do not independently guess counts.
- Do not count a single collaboration multiple times.
- Two concrete fine-policy classes participating in one Strategy collaboration count as ONE Strategy instance.
- source_version must be "original".

Output only the valid CSV in a code block.

CSV

```
source_version,summary,pattern_name,count
original,per_pattern,Model–View–Controller,1
original,per_pattern,Active Record,1
original,per_pattern,Template Method,1
original,per_pattern,GRASP Controller,1
original,per_pattern,GRASP Creator,1
original,per_pattern,GRASP Information Expert,2
original,total_pattern_types,,6
original,total_pattern_instances,,7
```

Based ONLY on the confirmed Part 1 design patterns, create a pattern architecture diagram.

The diagram must show:
- The relevant classes/interfaces/functions
- Their relationships
- Their pattern roles
- Which design pattern each collaboration represents

Do not include speculative patterns.

First provide Mermaid diagram code that I can save as the diagram source.

Then provide a short explanation of the diagram.

Do not modify the application source code.

Here is the Mermaid source for your **Part 1 Pattern Architecture Diagram**, based only on the seven confirmed instances in your `patterns.csv`. It shows the actual application participants, their relationships, and the corresponding pattern roles without including speculative patterns.

1. Mermaid diagram source

Save this as `Part1/pattern_architecture.mmd`.

2. Explanation of the diagram

The diagram separates the seven confirmed instances into distinct collaborations.

1. **MVC:** Shows routing, controllers, models, and views in the room-listing and booking flow. `ApplicationController` supplies shared controller setup.
2. **Active Record:** Shows `Room` and `Booking` inheriting from `ApplicationRecord`, which inherits from Rails' `ActiveRecord::Base`, and their corresponding database tables.
3. **Template Method:** Shows the shared `CrudConcern` included by `RoomsController`, `BookingsController`, and `SettingsController`. This is module-based reuse with resource-specific methods, not superclass inheritance.
4. **GRASP Controller:** Shows the cancellation route reaching `BookingsController#cancel`, which invokes `Booking#cancel!`.
5. **GRASP Creator:** Shows `RoomsController#init_booking_form` creating a booking through the room's `bookings` association.
6. **GRASP Information Expert — date availability:** Shows `Booking#dates_are_available` querying the room's bookings to check for overlapping reservations.
7. **GRASP Information Expert — price calculation:** Shows `Booking` including `HasPriceConcern`, which obtains the daily price and VAT information through associated objects.

Important: The diagram intentionally repeats some classes across subgraphs because the same class participates in multiple confirmed collaborations. These repetitions do not create additional pattern instances. The diagram is a source map of the confirmed roles and relationships, not a claim that all seven collaborations are independent subsystems.

No application source files have been modified.

Now begin Part 2: Design Pattern Evolution.

I will provide the original application and the requirements/implementation for Change 1, Change 2, and Change 3.

Do NOT immediately produce pattern_evolution.csv.

First analyze each change carefully.

For each change:

1. Explain the requirement.
2. Identify which source files were changed.
3. Identify which classes/interfaces/functions were added.
4. Identify which were modified.
5. Identify which were removed.
6. Identify which existing design-pattern collaborations are affected.
7. Identify whether any new pattern collaboration is introduced.
8. Compare the source before and after the change.
9. Determine the status of each affected pattern instance:
   - baseline
   - preserved
   - modified
   - extended
   - replaced
   - removed
   - newly_introduced

Do not infer intent that is not supported by the change/source.

For every conclusion, provide source evidence and exact file paths/line ranges.

Do not create CSV yet.

Worked for 7s

Understood. We'll handle **Part 2: Design Pattern Evolution** as a source-based before-and-after analysis, one change at a time. I will **not create** `pattern_evolution.csv` **yet**.

I'll use the confirmed Part 1 pattern inventory as the baseline and preserve the original application unchanged.

For each change, I'll document:

1. **Requirement** — what the change asks the application to do.
2. **Files changed** — exact paths.
3. **Added components** — new classes, modules, interfaces, methods, or functions.
4. **Modified components** — existing code whose behavior or structure changed.
5. **Removed components** — code actually deleted or no longer present.
6. **Affected Part 1 patterns** — which confirmed collaborations are affected and how.
7. **New patterns** — only collaborations supported by actual source evidence.
8. **Before-and-after comparison** — the original implementation versus the changed implementation.
9. **Pattern status** — `baseline`, `preserved`, `modified`, `extended`, `replaced`, `removed`, or `newly_introduced`.
10. **Evidence** — exact source paths and line ranges, with the relevant code behavior explained.

I'll distinguish **what the source proves from what can only be inferred**. A class being added or a method being renamed will not, by itself, be treated as proof of a new design pattern. If a change's implementation or supporting evidence is missing, I'll identify that gap instead of guessing.

Send Change 1

Please provide the **Change 1 requirement and its implementation**. You can upload the changed source files, a ZIP of the changed application, a patch/diff, or the implementation instructions.

I'll analyze Change 1 against the original baseline first, then follow the same process for Changes 2 and 3. We'll consolidate the findings only after all three changes have been examined.

well this is based on the prompt you should know right what are all the changes with the before prompts

Worked for 15s

You're right — I should have connected this to our earlier work on the **Rails Hotel Booking Design Pattern Case Study** instead of asking you to explain everything again. Sorry.

From our previous prompts, I can recover the four Part 2 requirements:

1. **Breakfast package** — introducing breakfast as an additional booking option.

2. **Airport transfer** — introducing airport transfer as an additional service.
3. **Room upgrade** — introducing the ability to upgrade a room.
4. **Cancellation policy** — introducing cancellation-policy-related behavior.

We had also established these important instructions in the earlier prompts:

- Analyze the **original GitHub repository first** and distinguish existing behavior from the new requirements.
- Explain the design impact of every requirement **before selecting or implementing patterns**.
- Choose proposed patterns based on concrete problems in the original design, rather than assuming a pattern must be used.
- Compare the baseline and proposed architecture, then prepare a file-by-file refactoring and implementation plan.
- Do not invent evidence or attribute proposed patterns to the original application.

How we should proceed with Part 2

For each change, I'll compare the original source against the corresponding changed implementation and document the exact files, added/modified/removed components, affected pattern collaborations, pattern statuses, and line-number evidence.

One detail I don't want to fabricate: I can recover all four requirements, but I don't have the exact original wording that maps them to `Change 1`, `Change 2`, and `Change 3`. I shouldn't arbitrarily combine two requirements into one change.

We can use the four requirements above as our established context. The next step is to inspect the actual Change 1, Change 2, and Change 3 implementation prompts or outputs to determine their exact mapping—without making you repeat the entire case study.

Now independently verify the Part 2 pattern evolution analysis against the actual original code and all three changed versions.

For every pattern instance and every source version:

1. Verify participants.
2. Verify source locations.
3. Verify the collaboration.
4. Verify its status after the change.
5. Verify the change description.
6. Check whether an instance was actually removed.

7. Check whether a new instance was actually introduced.
8. Check whether the same continuing collaboration retains the same instance_name.
9. Check for duplicate counting.
10. Check that removed instances are retained with status "removed".

Be conservative and source-based.

Produce a correction table before generating the final CSV.

Worked for 48s

Part 2 — Independent verification and correction table

I inspected the saved Part 2 files, all three change requirements and patches, the evolved Rails source, and the existing `pattern_evolution.csv`. The key correction is that **the current evolution CSV is incomplete and should not be used as the final dataset yet**.

The assignment requires every confirmed pattern collaboration to be tracked for each source version, with continuing collaborations retaining their original `instance_name` and removed instances retained with status `removed`.

1. Audit of the existing evolution CSV

Check	Finding	Required correction
Baseline inventory	The CSV includes only 4 of the 7 confirmed Part 1 instances.	Add MVC, Active Record, and GRASP Creator.
Instance names	The CSV uses `Generic CRUD Controller Workflow`, but the confirmed Part 1 name is `Shared Resource CRUD Workflow`.	Retain the exact Part 1 name for this continuing collaboration.
Source-version labels	The baseline label is `Original baseline 5e15551`; the later labels use `After Change1`, etc.	Use the assignment's version labels consistently: `original`, `after Change1`, `after Change2`, `after Change3`.
Change 1	The CSV records existing collaborations as preserved but does not record the effect of the new persisted `BookingService` model and `booking_services` table on the Active Record collaboration.	Update the Active Record instance to include the new persistence participants.

Change 2	The CSV does not record the new `RoomUpgrade` service in its effect on the existing price-calculation collaboration.	Evaluate the reuse of `compute_price_ht` and `compute_price_ttc` and distinguish it from the separate availability check.
Change 3	The Strategy row identifies the new policy classes, but it does not show whether the existing cancellation route actually uses the new service.	Record the Strategy collaboration as new, while explicitly noting that the existing controller cancellation action still calls `Booking#cancel!` directly.
Removed instances	No removed rows are present.	Keep this under review against each patch; do not invent removed instances. The supplied changes show no confirmed removal of the seven baseline collaborations.
Duplicate counting	The current CSV contains four continuing collaborations in each version and one Strategy row in Change 3, but omits three baseline instances.	Track one row per collaboration per version, not one row per participant class or per observation.

2. Corrected pattern-instance status matrix

The following is the corrected inventory to use when preparing the eventual CSV. It preserves the seven confirmed Part 1 instance names and counts each collaboration only once per version.

Confirmed pattern instance	`original`	`after Change1`	`after Change2`	`after Change3`
MVC — Room Listing and Booking Request Flow	`baseline`	`preserved`	`preserved`	`preserved`
Active Record — Room and Booking Persistence	`baseline`	`extended`	`extended`	`preserved`
Template Method — Shared Resource CRUD Workflow	`baseline`	`preserved`	`preserved`	`preserved`
GRASP Controller — Booking Cancellation Request Handling	`baseline`	`preserved`	`preserved`	`preserved`
GRASP Creator — Initialize a Booking for a Room	`baseline`	`preserved`	`preserved`	`preserved`
GRASP Information Expert — Booking Date Availability Validation	`baseline`	`preserved`	`preserved`	`preserved`
GRASP Information Expert — Booking Price Calculation	`baseline`	`preserved`	`extended`	`preserved`
Strategy — Cancellation Refund Policy	—	—	—	`newly_introduced`

Count check: 7 baseline collaborations; 7 continuing collaborations after each change; 1 additional Strategy collaboration after Change 3. No confirmed pattern collaboration is removed by the supplied changes.

The Active Record collaboration is marked `extended` after Change 1 because `Booking` gains a `has_many :booking_services` association and a new persistent model/table. After Change 2, the room-upgrade migration and `RoomUpgrade#call` add another persistence-related operation. The Strategy collaboration is new only in Change 3.

3. Evidence-based verification by change

Change 1 — Breakfast package and airport transfer

Requirement: Introduce optional services through `AdditionalService`, persist selections and quantities in `BookingService`, calculate additional-service totals independently, and add the booking-services table.

Finding	Source evidence	Verification
`AdditionalService` added	`Part2/code/app/models/additional_service.rb:1-12`	Defines the common name, unit price, and `price` behavior.
Concrete service classes added	`Part2/code/app/models/breakfast_package.rb:1-5`; `Part2/code/app/models/airport_transfer.rb:1-5`	Both inherit from `AdditionalService`; neither provides evidence of a separate Strategy collaboration.
Persistence model added	`Part2/code/app/models/booking_service.rb:1-11`	Associates a service record with a booking and calculates its subtotal.
Pricing service added	`Part2/code/app/services/additional_service_pricing.rb:1-5`	Aggregates service subtotals. This is a pricing utility, not sufficient evidence of a recognized design pattern on its own.
Booking persistence extended	`Part2/code/app/models/booking.rb:10-16`	`has_many :booking_services, dependent: :destroy` adds the association.
Database table added	`Part2/code/db/migrate/20261004170000_create_booking_services.rb:1-12`	Defines `booking_services` with a booking reference, service type, quantity, and unit price.

Pattern correction: Active Record should be `extended`. The existing MVC flow, shared CRUD workflow, cancellation handling, booking initialization, date-availability validation, and existing booking price-calculation collaboration remain in place.

Important limitation: The supplied files do not show the additional-service total being integrated into `Booking#amount_to_pay` or the existing `HasPriceConcern` calculation. Do not claim that the booking's final payable total now includes these service charges unless additional source evidence demonstrates that integration.

Change 2 — Room upgrade

Requirement: Move a booking to an available room, calculate the nightly price difference, recalculate totals, and persist the change.

Finding	Source evidence	Verification
Upgrade service added	`Part2/code/app/services/room_upgrade.rb:1–36`	`RoomUpgrade` encapsulates availability checking, price difference calculation, room reassignment, recalculation, and saving.
Availability check added	`Part2/code/app/services/room_upgrade.rb:24–26`	`available?` checks overlapping bookings for the target room, excluding the current booking.
Price difference calculated	`Part2/code/app/services/room_upgrade.rb:28–31`	Computes the target-versus-current daily-price difference multiplied by the booking duration.
Existing price methods reused	`Part2/code/app/services/room_upgrade.rb:12–16` ; `Part2/code/app/models/concerns/has_price_concern.rb:8–26`	The service calls the existing `compute_price_ht` and `compute_price_ttc` methods after changing the room. This supports marking the existing price-calculation collaboration `extended` .
Migration added	`Part2/code/db/migrate/20261004171000_add_room_upgrade_fields_to_bookings.rb:1–5`	Adds a nullable `upgraded_from_room` reference to bookings.

Pattern correction: The existing date-availability collaboration stays `preserved` ; the upgrade service implements a separate target-room availability check. Do not count that duplicate availability logic as a second instance of the confirmed `Booking Date Availability Validation` collaboration.

Implementation caveat: The shown `RoomUpgrade#call` changes `@booking.room` before saving. It does not assign `upgraded_from_room_id` . Also, the `difference` is returned in the result, but the shown method does not separately apply that difference to the stored total; instead, it recalculates the full totals. These are source-level observations, not assumptions about intended behavior.

No new confirmed design pattern is established by the upgrade service alone.

Change 3 — Cancellation and refund policies

Requirement: Support interchangeable cancellation/refund rules through a policy abstraction, with the cancellation workflow delegating policy decisions and then cancelling the booking.

Finding	Source evidence	Verification
Policy abstraction added	`Part2/code/app/services/cancellation_policy.rb:1-9`	Defines `cancellable?` and `refund_amount` as methods that raise `NotImplementedError`.
Cancellation service added	`Part2/code/app/services/cancellation_service.rb:1-14`	Accepts a policy object, delegates eligibility and refund calculation, and calls `@booking.cancel!`.
Flexible policy added	`Part2/code/app/services/flexible_cancellation_policy.rb:1-9`	Permits cancellation and returns `booking.amount_to_pay`.
Partial-refund policy added	`Part2/code/app/services/partial_refund_cancellation_policy.rb:1-13`	Returns the configured fraction of the booking amount.
No-refund policy added	`Part2/code/app/services/no_refund_cancellation_policy.rb:1-9`	Returns zero.
Existing cancellation action remains	`Part2/code/app/controllers/bookings_controller.rb:6-14`	The action still calls `@object.cancel!` directly.

Pattern correction: `Cancellation Refund Policy` is one `Strategy` collaboration, with `CancellationService` as context, `CancellationPolicy` as the abstraction, and three concrete policies. The three concrete policy classes count as **one** Strategy instance, not three.

Integration limitation: The existing `BookingsController#cancel` action does not invoke `CancellationService` in the supplied source. Therefore, the Strategy collaboration exists in the code, but the evidence does not establish that the application's existing cancellation request flow now uses those policies. The existing GRASP Controller collaboration remains `preserved`, not `replaced`.

4. Removal and duplicate-counting audit

Audit item	Result
Confirmed Part 1 collaboration removed by Change 1	None evidenced
Confirmed Part 1 collaboration removed by Change 2	None evidenced
Confirmed Part 1 collaboration removed by Change 3	None evidenced
Newly introduced confirmed pattern	Strategy — Cancellation Refund Policy, in Change 3

Additional-service classes counted as Strategy	No
Three cancellation policies counted as three Strategy instances	No; they form one collaboration
Separate upgrade availability method counted as a second existing Information Expert instance	No
Repeated appearances of a continuing instance across versions counted as new instances	No

One source-management limitation remains: the saved `Part2/code` directory is the cumulative evolved snapshot, while the `changes/Change1`, `Change2`, and `Change3` folders contain requirements and patches. The patch sequence supports the change-by-change comparison, but separate full source snapshots for each intermediate version are not present in the inspected folder. The final CSV should record only evidence that can be grounded in the corresponding baseline or change patch.

Conclusion: Correct the evolution inventory to include all seven confirmed Part 1 instances in every applicable version, retain their exact instance names, mark Active Record as extended after Changes 1 and 2, mark Booking Price Calculation as extended after Change 2, and introduce exactly one Strategy instance after Change 3. No `pattern_evolution.csv` has been generated.

Now generate the final pattern_evolution.csv based ONLY on the verified Part 2 analysis.

Use exactly these columns:

source_version
pattern_name
pattern_type
instance_name
participant_roles
source_locations
intent_and_evidence
status_after_change
change_description

Requirements:

- One row = one confirmed pattern collaboration in one source version.

- Include the original baseline.
- Include versions after Change1, Change2, and Change3 where applicable.
- Keep the SAME instance_name for a continuing collaboration.
- Retain removed instances with status "removed".
- Include all applicable participants.
- Use actual source paths and line ranges.
- Do not invent roles or evidence.
- Do not count before/after observations as separate instances.

Output only valid UTF-8 CSV inside a code block.

Worked for 11s

csv

```
source_version,pattern_name,pattern_type,instance_name,participant_roles,source_locations,intent_and_evidenc
original,Model–View–Controller,Architectural,Room Listing and Booking Request Flow,"Routing: Rails routes; C
original,Active Record,Data Access,Room and Booking Persistence,"Framework base: ActiveRecord::Base; Abstrac
original,Template Method,Behavioral,Shared Resource CRUD Workflow,"Shared CRUD implementation and hook metho
original,GRASP Controller,GRASP,Booking Cancellation Request Handling,"Request route: booking cancellation r
original,GRASP Creator,GRASP,Initialize a Booking for a Room,"Initiating client: RoomsController#init_bookin
original,GRASP Information Expert,GRASP,Booking Date Availability Validation,"Information expert and validat
original,GRASP Information Expert,GRASP,Booking Price Calculation,"Responsibility host: Booking; Calculation
after Change1,Model–View–Controller,Architectural,Room Listing and Booking Request Flow,"Routing: Rails rout
after Change1,Active Record,Data Access,Room and Booking Persistence,"Framework base: ActiveRecord::Base; Ab
after Change1,Template Method,Behavioral,Shared Resource CRUD Workflow,"Shared CRUD implementation and hook
after Change1,GRASP Controller,GRASP,Booking Cancellation Request Handling,"Request route: booking cancellat
after Change1,GRASP Creator,GRASP,Initialize a Booking for a Room,"Initiating client: RoomsController#init_b
after Change1,GRASP Information Expert,GRASP,Booking Date Availability Validation,"Information expert and va
after Change1,GRASP Information Expert,GRASP,Booking Price Calculation,"Responsibility host: Booking; Calcul
after Change2,Model–View–Controller,Architectural,Room Listing and Booking Request Flow,"Routing: Rails rout
after Change2,Active Record,Data Access,Room and Booking Persistence,"Framework base: ActiveRecord::Base; Ab
after Change2,Template Method,Behavioral,Shared Resource CRUD Workflow,"Shared CRUD implementation and hook
after Change2,GRASP Controller,GRASP,Booking Cancellation Request Handling,"Request route: booking cancellat
after Change2,GRASP Creator,GRASP,Initialize a Booking for a Room,"Initiating client: RoomsController#init_b
```

Using the verified pattern_evolution records, generate the pattern_counts.csv required for Part 2.

Count each source version separately.

For each source_version:
- provide per_pattern counts
- provide total_pattern_types
- provide total_pattern_instances

Important:
- Exclude removed instances from the current totals for that source version.
- Do not double-count the same collaboration.
- Use the exact pattern instances from pattern_evolution.csv.
- Do not independently reinterpret the source.

Output only the CSV.

CSV

```csv
source_version,summary,pattern_name,count
original,per_pattern,Model–View–Controller,1
original,per_pattern,Active Record,1
original,per_pattern,Template Method,1
original,per_pattern,GRASP Controller,1
original,per_pattern,GRASP Creator,1
original,per_pattern,GRASP Information Expert,2
original,total_pattern_types,,6
original,total_pattern_instances,,7
after Change1,per_pattern,Model–View–Controller,1
after Change1,per_pattern,Active Record,1
after Change1,per_pattern,Template Method,1
after Change1,per_pattern,GRASP Controller,1
after Change1,per_pattern,GRASP Creator,1
after Change1,per_pattern,GRASP Information Expert,2
after Change1,total_pattern_types,,6
after Change1,total_pattern_instances,,7
after Change2,per_pattern,Model–View–Controller,1
after Change2,per_pattern,Active Record,1
after Change2,per_pattern,Template Method,1
after Change2,per_pattern,GRASP Controller,1
```

Now begin Part 3: Cross-language comparison.

Based ONLY on the confirmed patterns from Part 1/Part 2, identify the pattern types that need cross-language implementations.

For each confirmed pattern type:

1. Choose ONE representative confirmed source collaboration.
2. Give it a meaningful instance/problem name.
3. Define a small, deterministic problem based on that pattern.
4. Define identical inputs.
5. Define identical expected outputs.
6. Explain the pattern roles.
7. Explain what must remain conceptually equivalent across:
   - Java
   - Python
   - JavaScript
   - C++

Do not write the implementations yet.

Do not invent a pattern that was not confirmed in the original application/evolution analysis.

Part 3 — Cross-language comparison plan

Based only on the confirmed Part 1 and Part 2 inventory, there are **7 distinct pattern types** to implement in Java, Python, JavaScript, and C++.

The two GRASP Information Expert instances count as one pattern type for this purpose. The three cancellation policy classes form one Strategy collaboration, not three instances.

The goal is to solve the same small problem in all four languages, using identical test inputs and expected outputs. The implementations should demonstrate the confirmed pattern's essential collaboration—not reproduce the entire Rails application or introduce additional patterns.

1. Confirmed pattern types and representative collaborations

Pattern type	Representative instance/problem	Source basis

Architectural — MVC	Room Listing and Booking Request Flow	Part 1: routes, controllers, models, views
Data Access — Active Record	Room and Booking Persistence	Part 1: `ApplicationRecord`, `Room`, `Booking`
Behavioral — Template Method	Shared Resource CRUD Workflow	Part 1: `CrudConcern` and its resource-specific controller methods
GRASP Controller	Booking Cancellation Request Handling	Part 1: `BookingsController#cancel` and `Booking#cancel!`
GRASP Creator	Initialize a Booking for a Room	Part 1: `RoomsController#init_booking_form` and `Room#bookings`
GRASP Information Expert	Booking Date Availability Validation	Part 1: `Booking#dates_are_available`
Behavioral — Strategy	Cancellation Refund Policy	Part 2, after Change 3: `CancellationService` and its three policy implementations

For the Information Expert type, I'll use the **date-availability** collaboration as the single representative problem. The price-calculation collaboration remains confirmed in the inventory, but a second cross-language implementation is not necessary when selecting one representative per pattern type.

2. Define the seven deterministic problems

1. MVC — Room Listing and Booking Request Flow

Part 1 confirmed

Problem: Given a list of rooms, handle a room-listing request and render the room titles.

Pattern roles

- **Model:** `Room` — holds room information.
- **Controller:** `RoomsController` — coordinates the request and obtains room data.
- **View:** `rooms/index.html.erb` — renders the room listing.

Identical input

JSON

```
{
  "action": "list_rooms",
```

```
  "rooms": [
    {"id": 1, "title": "Sea View"},
    {"id": 2, "title": "Garden Room"}
  ]
}
```

Expected output

JSON

```
{
  "room_titles": ["Sea View", "Garden Room"]
}
```

Conceptual equivalence across languages: Keep the model data separate from request coordination and presentation. The controller must obtain the room data and pass it to the view or equivalent presentation function. A single function that directly processes and prints everything would not demonstrate the same separation of responsibilities.

2. Active Record — Room and Booking Persistence

Part 1 confirmed; extended in Part 2

Problem: Create a room record, save it, and retrieve it by its identifier.

Pattern roles

- **Model:** `Room` — represents a room record and exposes persistence operations.
- **Persistence base:** The Rails source uses `ApplicationRecord`, inheriting from `ActiveRecord::Base`.
- **Stored record:** The room data that can be retrieved after saving.

Identical input

JSON

```
{
  "operation": "create_and_find",
  "room": {
    "id": 1,
```

```
      "title": "Sea View",
      "day_price": 150
    },
    "lookup_id": 1
  }
```

Expected output

JSON

```
{
  "found": true,
  "room": {
    "id": 1,
    "title": "Sea View",
    "day_price": 150
  }
}
```

Conceptual equivalence across languages: The model should represent the record and provide a save-and-retrieve interaction. To keep the test deterministic, the cross-language examples can use a small in-memory store. This demonstrates the record-oriented persistence behavior; it does **not** claim that Java, Python, JavaScript, or C++ has Rails' Active Record framework built in.

3. Template Method — Shared Resource CRUD Workflow

Part 1 confirmed

Problem: Run a shared create-record workflow for two resource types, with resource-specific validation and record construction.

Pattern roles

- **Shared workflow:** `CrudConcern#create`.
- **Resource-specific participants:** `RoomsController` and `BookingsController`.
- **Hook methods:** Resource-specific methods such as `object_class`, `model_params`, and `base_class`.

The original implementation uses `ActiveSupport::Concern` and method specialization, rather than a superclass-based template method.

Identical input

JSON

```
{
  "resource": "room",
  "record": {
    "title": "Sea View",
    "day_price": 150
  },
  "valid": true
}
```

Expected output

JSON

```
{
  "status": "created",
  "resource": "room",
  "title": "Sea View"
}
```

Conceptual equivalence across languages: There must be a shared workflow that controls the order of steps, while resource-specific behavior supplies the appropriate construction and validation. The language implementations may use inheritance, interfaces, composition, or callable hooks, but must retain the shared-algorithm/customization relationship. Using inheritance in another language should not be misrepresented as the exact implementation mechanism used by the Rails source.

4. GRASP Controller — Booking Cancellation Request Handling

Part 1 confirmed

Problem: Process a cancellation request for an existing booking.

Pattern roles

- **System-operation controller:** `BookingsController#cancel`.
- **Domain object:** `Booking`.
- **State-changing operation:** `Booking#cancel!`.

Identical input

```json
{
  "booking_id": 42,
  "action": "cancel",
  "authorized": true,
  "currently_canceled": false
}
```

Expected output

```json
{
  "status": "canceled",
  "booking_id": 42
}
```

Conceptual equivalence across languages: The controller receives and coordinates the request, checks authorization, invokes the booking's cancellation operation, and returns the result. The controller must not take over all domain-state manipulation itself.

For this small test, the input specifies an authorized, active booking so all four implementations follow the same successful path.

5. GRASP Creator — Initialize a Booking for a Room

Part 1 confirmed

Problem: Initialize a booking through the room to which it belongs.

Pattern roles

- **Initiating client:** `RoomsController#init_booking_form` .
- **Creation context:** `Room#bookings` .
- **Created object:** `Booking` .

Identical input

JSON

```
{
  "room_id": 7,
  "room_title": "Sea View",
  "user_id": 12,
  "num_guests": 1
}
```

Expected output

JSON

```
{
  "room_id": 7,
  "user_id": 12,
  "num_guests": 1
}
```

Conceptual equivalence across languages: A room's associated-bookings collection should provide the creation context and initialize the booking with the room relationship and supplied booking attributes. Do not replace this with an unrelated global factory and then claim it reproduces the original association-based collaboration.

6. GRASP Information Expert — Booking Date Availability Validation

Part 1 confirmed

Problem: Determine whether a proposed booking overlaps any existing booking for the same room.

Use half-open date intervals: the start date is included and the end date is excluded. This makes a booking ending on a date compatible with another booking beginning on that date.

Pattern roles

- **Information expert:** `Booking#dates_are_available` .
- **Related information provider:** `Room#bookings` .
- **Validation host:** `Booking` .

Identical input

JSON

```
{
  "room_id": 7,
  "proposed_start": "2026-11-10",
  "proposed_end": "2026-11-13",
  "existing_bookings": [
    {
      "id": 10,
      "start": "2026-11-11",
      "end": "2026-11-14"
    },
    {
      "id": 11,
      "start": "2026-11-13",
      "end": "2026-11-15"
    }
  ],
  "current_booking_id": null
}
```

Expected output

JSON

```
{
  "available": false
}
```

Conceptual equivalence across languages: The booking-validation responsibility uses the proposed dates and the room's existing bookings to determine overlap. Every implementation must apply the same interval rule and exclude the current booking when an existing booking is being edited. In this input, both existing reservations conflict with the proposed interval.

7. Strategy — Cancellation Refund Policy

Introduced in Part 2, Change 3

Problem: Calculate a refund by selecting a cancellation policy without changing the cancellation service's overall algorithm.

Pattern roles

- **Context:** `CancellationService`.
- **Policy abstraction:** `CancellationPolicy`.
- **Concrete strategies:** `FlexibleCancellationPolicy`, `PartialRefundCancellationPolicy`, and `NoRefundCancellationPolicy`.

These three concrete policies form **one Strategy collaboration**.

Identical input test set

JSON

```
[
  {"booking_amount": 1000, "policy": "flexible"},
  {"booking_amount": 1000, "policy": "partial"},
  {"booking_amount": 1000, "policy": "none"}
]
```

Expected output

JSON

```
[
  {"policy": "flexible", "refund_amount": 1000},
  {"policy": "partial", "refund_amount": 500},
```

```
    {"policy": "none", "refund_amount": 0}
  ]
```

The partial-refund test uses a 50% fraction for this cross-language example.

Conceptual equivalence across languages: The context must delegate refund calculation to the selected policy through a common contract. Changing the policy must change the calculation without requiring a different cancellation-service algorithm. The same three cases, policy names, and amounts must be tested in all four languages.

The cross-language example demonstrates the Strategy collaboration found in the Change 3 source. It does not imply that the existing Rails `BookingsController#cancel` action has been integrated with `CancellationService`.

3. What must remain identical across all four languages?

Requirement	Rule for Java, Python, JavaScript, and C++
Problem definition	Same seven representative problems and responsibilities.
Inputs	Identical test data, values, dates, identifiers, and policy selections.
Expected outputs	Identical logical results, regardless of language-specific formatting.
Pattern roles	Preserve the same responsibilities and collaboration between participants.
Behavior	Apply the same validation, overlap, creation, cancellation, and refund rules.
Test coverage	Run the same cases in every language, including the three Strategy policies.
Implementation details	Use idiomatic language constructs without changing the problem's design intent.
Source evidence	Record the relevant confirmed Rails collaboration as the representative source instance; keep the cross-language snippets separate from production source.

Important boundary: MVC and Active Record are framework-oriented patterns in the original Rails application. Their small cross-language examples are conceptual equivalents, not implementations of Rails itself. Likewise, the cross-language Template Method example must preserve the shared workflow and hooks while acknowledging that the original uses a concern.

No implementation code has been written yet. The next step is to create the same test specification for each language and then implement and verify each representative collaboration against it.

For each confirmed Part 3 pattern problem, now create four equivalent implementations:

1. Java
2. Python
3. JavaScript
4. C++

Requirements:

- Same problem.
- Same input.
- Same expected output.
- Same conceptual pattern collaboration.
- Keep the implementations small and easy to test.
- Production snippet and test runner must remain conceptually separate.
- Do not include tests in the production source-location measurement.
- Clearly identify the production code.
- Provide a test runner for each language.
- Provide compile/run/test commands.
- State the required runtime/compiler.
- Do not claim a test passed unless it has actually been executed.

Do NOT fabricate runtime measurements.

Analyzed

Analyzed

Created the cross-language implementation package with all **7 confirmed pattern problems implemented in Java, Python, JavaScript, and C++**.

Download Part 3 — Cross-language implementations and tests

The ZIP includes:

- **28 production source files —** one for each pattern-language combination.
- **28 separate test runners —** kept separate from production code for source-location measurements.

- `specification.json` — shared inputs and expected outputs.
- `README.md` — runtime requirements and compile/run/test commands.
- `production_files_manifest.txt` — identifies production files separately.
- `execution_report.txt` — records actual test execution results.

Verification results

28 of 28 test runners passed

All seven patterns tested in each of the four languages.

Python	JavaScript
7/7 passed	**7/7 passed**
Java	C++
7/7 passed	**7/7 passed**

These results are from actually compiling or executing the test runners in the available environment. They are **functional test results, not runtime performance measurements**.

One important distinction for your report: these are small cross-language demonstrations of the confirmed collaborations, not claims that the original Rails application uses identical implementation mechanisms in all four languages.

‹/›

Now prepare the Part 3 verification procedure.

For every language implementation:

1. Compile first where applicable.
2. Run one warmup.
3. Run five measured runs.
4. Use the same machine/environment.
5. Record actual elapsed times.
6. Calculate the median.
7. Record actual passed/total test results.
8. Record runtime/compiler name and version.
9. Clearly state what the timing includes.

10. Keep raw timing/test results as evidence.

Do NOT estimate or fabricate any execution time.

If you cannot actually execute a language in this environment, mark the timing as blank/not_run and explain why.

Create the exact evidence files I should save for each language.

Analysis errored

Analysis errored

‹/›

Now generate the final cross_language.csv using ONLY the actual verified Part 3 results.

Use exactly these columns:

pattern_name
instance_name
language
snippet_path
source_loc
max_function_cc
runtime_compiler
test_command
test_result
median_process_ms
language_notes

Requirements:

- Use actual production snippet paths.
- source_loc must exclude blank lines, comments and tests.
- max_function_cc must use the complexity counting rule/tool documented in README.
- runtime/compiler must be actual.

- test_command must be reproducible.
- test_result must be actual.
- median_process_ms must be actual or blank if not run.
- Never estimate timing.
- language_notes should describe genuine language-specific structural/static/dynamic differences.

Output only valid CSV.

Worked for 30s

Analyzed

‹/›

Now create the llm.csv information for this part of the assignment.

Use exactly these columns:

part
model_and_date
prompt_file
input_file
output_file
evaluation
verification_file

For every AI interaction used for:
- pattern discovery
- evolution analysis
- cross-language analysis

record:
- the displayed model name
- actual interaction date
- exact prompt file path
- exact input file path
- raw output file path
- evaluation: Correct / Partial / Incorrect / Cannot confirm

- concise correction
- source/test verification file

Do not invent dates, filenames, or verification evidence.

If a value is unknown, explicitly mark it as UNKNOWN rather than guessing.

Analyzed

Analyzed

‹/›

llm.csv
Spreadsheet

Now create the README.md for my complete Design Pattern Case Study repository.

Follow the official assignment instructions exactly.

Include:

1. Team information
2. Student roll numbers, names and batch
3. Application/source URL
4. Source commits
5. Source versions
6. Environment versions
7. Run commands
8. Test commands
9. Inspected modules
10. Design-pattern analysis methodology
11. Pattern verification methodology
12. LOC rules
13. Cyclomatic complexity counting rule/tool
14. Cross-language timing methodology
15. Timing inclusion/exclusion rules

16. Runtime/compiler versions
17. Limitations
18. LLM usage methodology
19. How raw prompts and outputs are stored
20. How source evidence was used to verify LLM results
21. Repository structure
22. Reproducibility instructions

Do not invent information that is not known from the source/project.

Use placeholders such as [TEAM NAME] or [COMMIT HASH] where I need to provide information manually.

Worked for 25s

Analyzed

‹/›

README.md
Document

```
A1/
│
├── README.md
├── A1_Report.pdf
├── A1_Report.tex
│
├── data/
│
├── code/
│   └── original application source
│
├── test_result/
│   └── test cases / screenshots / results
│
├── Part1/
│   ├── patterns.csv
│   ├── Pattern_count.csv
```

```
│   ├── pattern_architecture/
│   │   ├── diagram source
│   │   └── diagram image
│   ├── Part1_prompt.txt
│   └── llm.csv
│
├── Part2/
│   ├── code/
│   │   └── evolved application source
│   ├── changes/
│   │   ├── Change1/
│   │   │   ├── requirement
│   │   │   ├── diff
│   │   │   └── test result
│   │   ├── Change2/
│   │   └── Change3/
│   ├── pattern_evolution.csv
│   ├── Pattern_count.csv
│   ├── diagrams/
│   ├── Part2_prompt.txt
│   └── llm.csv
│
└── Part3/
    ├── <pattern_problem_name>/
    │   ├── java/
    │   ├── python/
    │   ├── javascript/
    │   └── cpp/
    │
    ├── llm/
    │   ├── part3_prompt.txt
    │   ├── llm.csv
    │   └── pattern_crosslanguage.csv
    │
    └── A1_slide.pdf
```

FINAL PRE-SUBMISSION AUDIT

Act as a strict course-assignment auditor.

Compare my actual repository/team folder against EVERY requirement in the uploaded Design Pattern Case Study Instructions PDF.

Do not judge based on our previous conversation. Inspect the actual files.

Create a table:

Required artifact | Required by instructions? | Exists? | Correct location? | Correct format? | Content verified? | Problem | Action required

Check:
- README
- report PDF
- LaTeX source
- original code
- evolved code
- test evidence
- Part 1 CSVs
- Part 1 diagram/source
- Part 1 prompt
- Part 1 llm.csv
- Change1/2/3 evidence
- Part 2 CSVs
- Part 2 diagrams
- Part 2 prompt
- Part 2 llm.csv
- Java snippets
- Python snippets
- JavaScript snippets
- C++ snippets

- test runners
- language READMEs
- raw test results
- timing evidence
- Part 3 prompt
- Part 3 llm.csv
- cross-language CSV
- slides PDF

Also check:
- CSV column names
- UTF-8 encoding
- quoted CSV fields where necessary
- relative paths
- source locations
- pattern counts
- removed pattern handling
- newly introduced pattern handling
- actual test results
- actual timing measurements
- reproducibility
- no passwords/secrets
- no compiled binaries/dependency caches

Do NOT mark an item PASS unless you can actually verify it.

Finish with:

1. READY TO SUBMIT
2. MUST FIX
3. OPTIONAL IMPROVEMENTS

Do not submit or upload anything to GitHub yourself.

these are the all files i need to submit makesure you generate

everything and make it in to one zip file and if the files are generated already just give them if not just create them and give all in one zip

Worked for 2m 24s